

Multi-Head Attention.

One attention head sees the world through a single lens. Multi-head attention runs h lenses in parallel on complementary subspaces — for the *same* parameter budget. We derive the equations, prove the “no extra cost” parameter count, and compute a complete two-head example by hand.

THE EQUATIONS

$$\begin{aligned}\text{MultiHead}(X) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O, \\ \text{head}_i &= \text{Attention}(XW_Q^i, XW_K^i, XW_V^i).\end{aligned}$$

Worked example. $d_{\text{model}} = 4$, $h = 2$ heads ($d_k = 2$), sequence length $T = 3$. Every projection, head, concatenation, and output mix is multiplied out in full.

Topic 4 of 5 in this module · companion to the M0.1 course page · v1.0

Contents

1	Why more than one head	2
2	The data flow	3
3	Parameter count — multi-head is “free”	3
4	Worked example — two heads by hand	3
5	Practical notes	5
A	Weight matrices (reproducible)	5

1 Why more than one head

A single attention head produces *one* set of weights per query — one way of deciding what is relevant. But a token relates to its context in many simultaneous ways: syntactic agreement, coreference, local adjacency, topical similarity. Forcing all of these through one softmax means they must compromise into a single attention pattern.

Multi-head attention runs h independent attention operations (“heads”) in parallel, each on its own learned $d_k = d_{\text{model}}/h$ dimensional subspace, then concatenates and mixes the results:

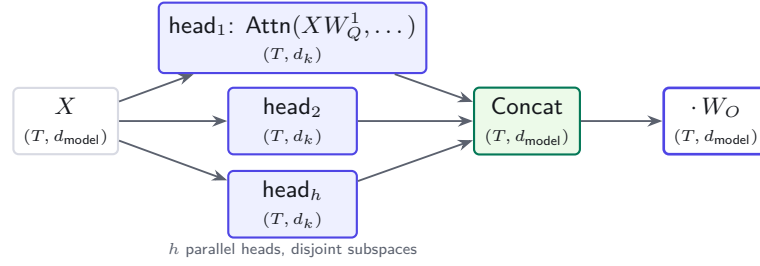
$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O, \quad (6)$$

$$\text{head}_i = \text{Attention}(XW_Q^i, XW_K^i, XW_V^i). \quad (7)$$

Here $X \in \mathbb{R}^{T \times d_{\text{model}}}$ is the input, $W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{d_{\text{model}} \times d_k}$ project into head i ’s subspace, and $W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ recombines the concatenated heads.

Think of h specialists reading the same sentence. One tracks subject–verb agreement, another links pronouns to their referents, another just looks at neighbours. Each writes a short report (d_k numbers per token); W_O is the editor that fuses the reports into one representation. Probing studies (Clark et al., 2019) confirm real BERT heads specialise this way.

2 The data flow



The split into heads is a *reshape*, not a new computation: the projection matrices W_Q, W_K, W_V together still map $d_{\text{model}} \rightarrow d_{\text{model}}$; we merely slice the output into h blocks of width d_k .

3 Parameter count — multi-head is “free”

The surprising fact: h heads use *exactly the same* number of parameters as one big head of width d_{model} .

Single head (width $d_{\text{model}} = d$): three projections $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ plus output $W_O \in \mathbb{R}^{d \times d}$, total

$$4d^2.$$

Multi-head (h heads, each $d_k = d/h$): each head's $W_Q^i \in \mathbb{R}^{d \times d_k}$ has $d \cdot d_k$ entries; summed over h heads,

$$\sum_{i=1}^h d \cdot d_k = h \cdot d \cdot \frac{d}{h} = d^2.$$

The same holds for K and V projections, plus $W_O \in \mathbb{R}^{d \times d}$, giving $3d^2 + d^2 = 4d^2$ — identical.

Quantity	Single head ($d=768$)	12 heads ($d_k=64$)
Q projection params	$768^2 = 589,824$	$12 \times 768 \times 64 = 589,824$
all four projections	$4 \times 768^2 = 2,359,296$	$2,359,296$
attention patterns learned	1	12

Same parameter budget, $h \times$ the representational diversity. Multi-head attention is one of the rare “free lunch” design choices: splitting a wide projection into h narrow ones costs nothing in parameters or (with batched matmuls) compute, yet lets the layer attend to h different relationships at once.

4 Worked example — two heads by hand

We use $d_{\text{model}} = 4$, $h = 2$ (so $d_k = 2$), $T = 3$ tokens. Input and projection weights (Appendix A):

$$X = \begin{bmatrix} 1.000 & -0.428 & 0.626 & 0.323 \\ 0.597 & 0.524 & -0.989 & 0.913 \\ 0.405 & 0.968 & 0.161 & 1.137 \end{bmatrix}.$$

Projecting $Q = XW_Q$, $K = XW_K$, $V = XW_V$ gives

$$Q = \begin{bmatrix} 0.8801 & 0.3161 & 1.1761 & 1.0249 \\ -0.2404 & -0.7087 & 0.1155 & 0.0484 \\ -0.5794 & -0.2201 & 0.3447 & -0.4244 \end{bmatrix}, \quad K = \begin{bmatrix} -0.3995 & -0.1969 & 0.3476 & 0.6803 \\ -0.1779 & 1.2785 & -1.1398 & -0.8929 \\ -0.6101 & 0.4373 & -0.8516 & -0.6703 \end{bmatrix},$$

$$V = \begin{bmatrix} 0.1744 & -0.5648 & 0.2178 & 0.1717 \\ -1.0413 & 0.0520 & -0.2086 & -0.1487 \\ -0.3035 & -0.4200 & -0.0174 & -0.8353 \end{bmatrix}.$$

4.1 Split into heads

Columns $\{1, 2\}$ form head 1; columns $\{3, 4\}$ form head 2. Each head runs scaled dot-product attention with $d_k = 2$ (divisor $\sqrt{2} \approx 1.414$), exactly as in Topic 1.

4.2 Head 1

Using $Q_{1:2}$, $K_{1:2}$, $V_{1:2}$, the weights $A^{(1)} = \text{softmax}(QK^\top/\sqrt{2})$ and output $A^{(1)}V$:

$$A^{(1)} = \begin{bmatrix} 0.277 & 0.443 & 0.280 \\ 0.452 & 0.208 & 0.341 \\ 0.369 & 0.268 & 0.364 \end{bmatrix}, \quad A^{(1)}V_{1:2} = \begin{bmatrix} -0.4975 & -0.2513 \\ -0.2409 & -0.3874 \\ -0.3247 & -0.3471 \end{bmatrix}.$$

Head-1 attention heatmap (row = token querying, column = token attended to):

	tok1	tok2	tok3
tok1	0.28	0.44	0.28
tok2	0.45	0.21	0.34
tok3	0.37	0.27	0.36

4.3 Head 2

Using $Q_{3:4}$, $K_{3:4}$, $V_{3:4}$:

$$A^{(2)} = \begin{bmatrix} 0.812 & 0.075 & 0.113 \\ 0.370 & 0.310 & 0.320 \\ 0.309 & 0.345 & 0.346 \end{bmatrix}, \quad A^{(2)}V_{3:4} = \begin{bmatrix} 0.1592 & 0.0342 \\ 0.0102 & -0.2500 \\ -0.0106 & -0.2873 \end{bmatrix}.$$

Notice head 2 behaves *differently* from head 1: token 1 attends almost entirely to itself (0.81), whereas in head 1 token 1 spread its attention. Same input, two distinct patterns — the whole point.

	tok1	tok2	tok3
tok1	0.81	0.08	0.11
tok2	0.37	0.31	0.32
tok3	0.31	0.34	0.35

4.4 Concatenate and project

Glue the two head outputs side by side (back to width $d_{\text{model}} = 4$), then mix with W_O :

$$\text{Concat} = \begin{bmatrix} -0.4975 & -0.2513 & 0.1592 & 0.0342 \\ -0.2409 & -0.3874 & 0.0102 & -0.2500 \\ -0.3247 & -0.3471 & -0.0106 & -0.2873 \end{bmatrix} \xrightarrow{\cdot W_O} \text{MultiHead}(X) = \begin{bmatrix} -0.2846 & -0.1718 & 0.0554 & -0. \\ -0.1645 & -0.0499 & 0.3196 & -0. \\ -0.2316 & -0.0801 & 0.3134 & -0. \end{bmatrix}$$

Shape in = shape out = (3, 4), so this drops straight into the residual+LayerNorm of a transformer block (Module 0.2). W_O is what lets the heads *talk*: without it, head outputs would stay in separate slots.

5 Practical notes

1. **The scaling uses d_k , the per-head width** — here $\sqrt{2}$, not $\sqrt{4}$. In BERT-base ($d_{\text{model}} = 768, h = 12$) it is $\sqrt{64} = 8$ (see Topic 2’s pitfall).
2. **Heads are independent only inside attention.** They share nothing until W_O recombines them; W_O learns how much of each head to trust.
3. **Choosing h .** Too few heads under-specialise; too many make each d_k tiny and noisy. Common choices keep $d_k \in \{64, 128\}$: BERT-base $h=12$, GPT-3 $h=96$, LLaMA-7B $h=32$.
4. **Implementation.** All heads are computed in one batched matmul by reshaping $(T, d_{\text{model}}) \rightarrow (h, T, d_k)$ — the “parallel” heads never loop in code.

Concatenation order matters only in that W_O must be consistent with it, but a subtler bug is forgetting W_O entirely (or replacing it with identity). Without the output projection the model cannot mix information *across* heads, and multi-head collapses to h isolated low-rank attentions — usually a noticeable quality drop.

A Weight matrices (reproducible)

$$W_Q = \begin{bmatrix} 0.4 & 0.1 & 0.4 & 0.5 \\ -0.9 & 0.0 & -0.7 & -1.0 \\ 0.1 & 0.5 & 0.4 & 0.0 \\ 0.1 & -0.3 & 0.7 & 0.3 \end{bmatrix}, \quad W_K = \begin{bmatrix} 0.1 & 0.4 & 0.1 & 0.1 \\ 0.2 & 0.3 & -0.3 & -0.7 \\ -0.3 & -0.8 & 0.5 & 0.5 \\ -0.7 & 0.1 & -0.6 & -0.1 \end{bmatrix},$$

$$W_V = \begin{bmatrix} -0.8 & -0.4 & 0.2 & 0.5 \\ -0.8 & -0.2 & 0.1 & -0.2 \\ 0.7 & -0.4 & 0.2 & -0.3 \\ 0.6 & 0.0 & -0.2 & -0.7 \end{bmatrix}, \quad W_O = \begin{bmatrix} 0.7 & 0.3 & 0.1 & 0.6 \\ 0.0 & -0.2 & -0.5 & 0.0 \\ 0.4 & -0.5 & 0.0 & -0.1 \\ 0.0 & 0.2 & -0.6 & -0.3 \end{bmatrix}.$$

```
import numpy as np
def sm(a): e=np.exp(a-a.max(1,keepdims=True)); return e/e.sum(1,keepdims=True)
Q,K,V = X@WQ, X@WK, X@WV
def head(c):
    q,k,v = Q[:,c],K[:,c],V[:,c]
    A = sm(q@k.T/np.sqrt(2)); return A@v
MHA = np.hstack([head([0,1]), head([2,3])]) @ WO
```

Marevlo Research · Transformer Track · Module 0.1 “Attention Refresher” · Topic 4 of 5: Multi-Head Attention.